

ఇండియన్ ఇన్స్టిట్యూట్ ఆఫ్ ఇన్ఫర్మేషన్ టెక్నాలజీ,
డిజైన్ అండ్ మాన్యుఫ్యాక్చరింగ్, కర్నూలు



भारतीय सूचना प्रौद्योगिकी, अभिकल्पना
एवं विनिर्माण संस्थान, कर्नूल

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY,
DESIGN AND MANUFACTURING, KURNOOL
Jagannathagattu, Dinnevarapadu, Kurnool, Andhra Pradesh – 518008

Virtualization Lab: Performance and Isolation Analysis of KVM/QEMU, VirtualBox, and Native LXC

*A Project Report Submitted in Partial Fulfillment of the Requirements
for the Course of Operating Systems / Advanced Virtualization Lab*

Submitted By:

Chakala Karthik (123CS0038)

Shaik Venkat (123CS0037)

B.Tech, Computer Science and Engineering

Under the Guidance of:

Dr. Anil Kumar

Assistant Professor

Department of Computer Science and Engineering

IIITDM Kurnool

Academic Year 2025–2026

September 2026

Contents

1	Introduction and Background	2
2	Virtualization Concepts and Architectural Analysis	4
3	System Architecture and Benchmarking Methodology	6
4	Experimental Testbed and Configuration	8
5	Benchmark Methodology and Workload Implementation	9
6	Platform Implementation and Engineering Details	13
7	Empirical Results and Comparative Analysis	14
8	Management and Telemetry Web Dashboard	17
9	Discussion and Architectural Trade-off Space	19
10	Conclusion and Future Work	20

1 Introduction and Background

Virtualization constitutes one of the foundational architectural abstractions in modern computer engineering and cloud computing infrastructure. By decoupling operating system instances and computational workloads from physical hardware, virtualization technologies enable high-density server consolidation, multi-tenant isolation, elastic resource allocation, rapid provisioning, and disaster recovery. Over the past three decades, virtualization has evolved from legacy mainframe hardware partitioning into a diverse spectrum of heterogeneous technologies ranging from kernel-integrated hypervisors and hosted desktop hypervisors to lightweight operating system-level containers.

Despite the pervasive adoption of virtualized infrastructure, the abstraction layers introduced by virtualization inevitably impose performance trade-offs, commonly termed the *virtualization tax*. These penalties stem from guest-to-host privilege transitions (VM-exits), two-dimensional address translation (Extended Page Tables / Nested Page Tables), virtual device register trapping, interrupt injection latencies, and hypervisor scheduling contention. Understanding the precise empirical magnitude and architectural origins of these overheads is essential for systems architects, cloud engineers, and software developers when selecting an appropriate execution platform.

1.1 Evolution and Taxonomy of Virtualization

The historical taxonomy established by Popek and Goldberg [1] categorized virtualization architectures primarily based on the presence or absence of an intermediate host operating system:

1. **Type-1 (Bare-Metal) Hypervisors:** Hypervisors that execute directly on bare physical hardware in the most privileged execution mode, bypassing any general-purpose host operating system. Examples include VMware ESXi, Xen [3], and modern kernel-integrated architectures such as the Linux Kernel-based Virtual Machine (KVM) [9].
2. **Type-2 (Hosted) Hypervisors:** Virtual machine monitors (VMMs) that execute as standard user-space applications on top of a conventional host operating system. The host OS retains direct control over the physical CPU, memory, and devices. Prominent examples include Oracle VM VirtualBox [11] and VMware Workstation.
3. **Operating System-Level Virtualization (Containers):** Container runtimes that instantiate isolated user-space environments using host kernel primitives—specifically kernel namespaces and control groups (cgroups) [7, 8]—without hardware emulation or redundant guest kernels. Prominent implementations include Native Linux Containers (LXC) [10] and Docker.

With the advent of hardware-assisted virtualization extensions on x86_64 processors (Intel VT-x and AMD-V), the boundaries between traditional Type-1 and Type-2 classifications have become nuanced. In modern Linux systems, KVM turns the host kernel itself into a hypervisor via loadable kernel modules, allowing virtual machines to execute directly on physical processor cores in VMX non-root mode, while delegating peripheral

device emulation to a user-space helper process (QEMU) [4]. Conversely, Oracle VirtualBox utilizes an out-of-tree kernel driver (`vboxdrv`) to coordinate ring-0 context switching, but operates under a hosted process model. Meanwhile, native containers completely bypass CPU privilege trapping by sharing the underlying host kernel.

1.2 Research Objectives and Scope

The central objective of this research is to design, construct, and evaluate a comprehensive, reproducible, and non-destructive benchmarking laboratory platform—designated the **CC2 Virtualization Benchmarking Suite**—to conduct a rigorous empirical comparison between three leading virtualization platforms and an unencumbered bare-metal host baseline on identical physical hardware:

1. **Bare-Metal Host Baseline:** Physical Ubuntu 24.04.5 LTS host environment acting as the control reference.
2. **KVM/QEMU:** Hardware-accelerated kernel hypervisor utilizing QEMU 8.2.2 and `libvirt` 10.0.0.
3. **Oracle VirtualBox:** Hosted Type-2 hypervisor running version 7.2.16 with kernel drivers.
4. **Native LXC:** Native Linux Containers running version 5.0.3 utilizing Linux cgroups v2 and namespace isolation.

To systematically investigate the performance envelope across these environments, this investigation evaluates six primary research questions:

- **RQ1 (Compute Efficiency):** To what degree do hardware-assisted virtualization extensions eliminate CPU compute overhead during sustained floating-point arithmetic compared to bare-metal execution and containerization?
- **RQ2 (Memory Subsystem):** What is the measurable impact of two-dimensional paging (Extended Page Tables) on memory bandwidth, cache locality, and allocation overhead compared to native page tables?
- **RQ3 (Kernel Boundary and System Calls):** How do system call invocation and thread context-switch latencies differ when executed within a virtualized guest kernel versus a container sharing the host kernel?
- **RQ4 (Networking and Bridging):** What latency penalties are introduced by virtual software bridges (`virbr0`, `vboxnet0`, `lxcbr0`) and virtual network interface emulation (VirtIO versus virtual hardware controllers)?
- **RQ5 (Provisioning and Cold-Start):** What are the lifecycle costs associated with domain initialization, guest kernel bootstrapping, network socket readiness, and application availability?
- **RQ6 (Isolation Security Boundaries):** How do hypervisor hardware boundaries compare with kernel namespace and cgroup boundaries in terms of process isolation, device visibility, and cross-domain contention?

1.3 Academic and Scientific Integrity Policy

In accordance with strict scientific and academic standards, this report enforces three foundational methodological invariants:

1. **Zero Synthetic Data Fabrication:** All numerical metrics, execution timings, latency percentiles, and throughput figures presented in this report represent real measurements captured on the physical testbed. If an optional utility or hardware counter interface was unavailable (e.g., restricted access to hardware PMU registers under `perf_event_paranoid=4`, or absent packages such as `iperf3` and `fio`), the platform explicitly records the measurement as `UNAVAILABLE` or `FAILED` rather than fabricating zero or mock values.
2. **Deterministic Cryptographic Verification:** All custom microbenchmarks are compiled into self-contained static C99 binaries whose binary integrity is validated against immutable SHA-256 digests. Workload mathematical outputs are continuously verified using FNV-1a checksums to guarantee computational equivalence across all target environments.
3. **Strictly Descriptive Reporting:** In lieu of subjective scoring algorithms or declaring an arbitrary “winner”, this study characterizes the multidimensional trade-offs inherent to each architecture across performance, security, operational overhead, and flexibility.

1.4 Report Organization

The remainder of this report is structured as follows. Section 2 delivers an architectural analysis of hardware-assisted virtualization, KVM/QEMU, VirtualBox, and LXC. Section 3 details the system architecture of the CC2 benchmarking platform, execution lifecycle, and mathematical analysis framework. Section 4 specifies the physical host testbed, virtualized guest topologies, and benchmark parameters. Section 5 details the implementation of the ten microbenchmark suites. Section 7 presents the empirical measurements, comparative tables, and in-depth performance analysis. Section 8 details the telemetry web dashboard. Section 9 discusses architectural trade-offs, and Section 10 concludes the report with directions for future work.

2 Virtualization Concepts and Architectural Analysis

Understanding the empirical behavior of virtualized systems requires an architectural analysis of the abstraction layers through which CPU instructions, memory references, and I/O requests travel. This section analyzes hardware-assisted virtualization, the kernel-integrated model of KVM/QEMU, the hosted model of Oracle VirtualBox, and the operating system-level containerization model of Native LXC.

2.1 Hardware-Assisted CPU and Memory Virtualization

Early x86 processor architectures were notoriously difficult to virtualize due to seventeen sensitive, unprivileged instructions identified by Popek and Goldberg [1] (e.g., POPF, PUSHF, SGDT, SIDT, SMSW). When executed in Ring 1 or Ring 3 by a guest operating system, these instructions either failed silently or examined host hardware state without triggering a general protection fault (#GP), preventing classical trap-and-emulate virtualization. Hypervisors historically resolved this through complex binary translation (as pioneered by VMware) or paravirtualization (as implemented in Xen [3]).

2.1.1 Intel VT-x and VMX Operation

In 2005, Intel introduced hardware-assisted virtualization extensions (Intel VT-x) to eliminate the need for software binary translation. VT-x introduces two fundamental processor operating modes:

1. **VMX Root Operation:** The execution mode reserved for the hypervisor. In this mode, all CPU privilege rings (Ring 0 through Ring 3) operate with full hardware authority, identical to standard physical execution.
2. **VMX Non-Root Operation:** The execution mode designated for guest operating systems and their user applications. While the guest OS executes within Ring 0 of non-root mode, sensitive operations (such as modifying control registers CR0, CR3, CR4, executing CPUID, or servicing external physical hardware interrupts) trigger a mandatory hardware transition back to VMX root mode, known as a **VM-Exit**.

State transitions between root and non-root modes are governed by a physical memory-backed structure known as the **Virtual Machine Control Structure (VMCS)**. The VMCS contains:

- **Guest-State Area:** CPU registers (RIP, RSP, CR0/3/4, segment selectors) restored upon **VM-Entry** (VMLAUNCH or VMRESUME).
- **Host-State Area:** Hypervisor registers loaded automatically upon a VM-Exit.
- **VM-Execution Control Fields:** Bitmasks defining events that compel a VM-Exit (e.g., CR3 writes, I/O instructions, MSR accesses, interrupts).
- **VM-Exit Information Fields:** Read-only diagnostic data documenting the exact reason for the transition (EXIT_REASON) and qualification flags.

Every VM-Exit imposes an architectural latency penalty of several hundred CPU cycles required to serialize pipeline state, flush execution buffers, save guest registers to the VMCS, load host registers, and transfer execution to the hypervisor's exit handler.

2.1.2 Two-Dimensional Paging (Extended Page Tables)

CPU execution within a guest requires translating Guest Virtual Addresses (GVA) to physical memory. In a virtualized environment, physical memory is an abstraction: the guest translates GVA to a **Guest Physical Address (GPA)**, which the hypervisor must subsequently map to a physical **Host Physical Address (HPA)**.

To bypass costly software-based shadow page tables, modern x86 processors implement Second Level Address Translation (SLAT), termed **Extended Page Tables (EPT)** by Intel. Under EPT, the hardware Memory Management Unit (MMU) performs a two-dimensional page walk. For each of the four levels of guest page tables (PML4 → PDPT → PD → PT), the hardware MMU must perform a complete four-level EPT page walk to resolve the guest physical address of the page table pointer itself. Consequently, in the worst case where Translation Lookaside Buffer (TLB) entries miss at every stage, resolving a single memory reference requires:

$$N_{\text{accesses}} = 4 \times 4 + 4 = 24 \text{ memory references} \quad (1)$$

Although hardware TLBs, EPT TLB tagging (Virtual Processor IDs or VPIDs), and huge pages (2 MB and 1 GB) mitigate this penalty in steady-state execution, workloads characterized by high TLB churn, pointer chasing, or dynamic working sets experience measurable memory throughput degradation under EPT.

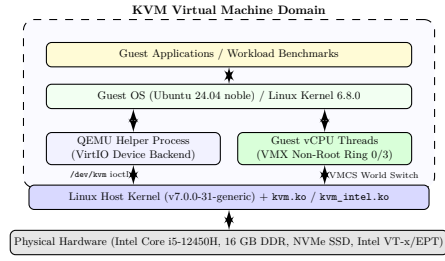


Figure 1: KVM/QEMU Architecture.

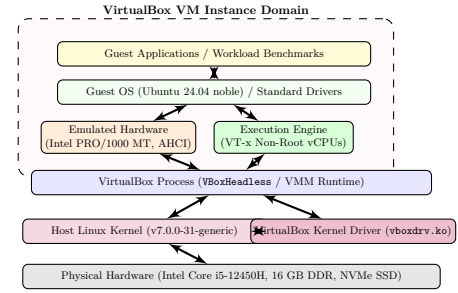


Figure 2: VirtualBox Hosted Architecture.

2.2 KVM/QEMU Architectural Model

The Kernel-based Virtual Machine (KVM) [9] represents an in-tree Linux kernel virtualization module that effectively transforms the Linux host kernel into a Type-1-like hypervisor. Rather than running as an external microkernel beneath Linux, KVM is implemented as loadable kernel modules:

- **kvm.ko**: Core hardware-independent virtualization infrastructure, vCPU management, memory slot tracking, and event routing.
- **kvm_intel.ko**: Architecture-specific driver managing Intel VMX hardware instructions, VMCS initialization, and VM-exit decoding.

The hypervisor interfaces with user space through the character device node `/dev/kvm`. A management process, QEMU [4], issues `ioctl()` system calls to configure virtual machines: `KVM_CREATE_VM` instantiates the VM address space, `KVM_SET_USER_MEMORY_REGION` sets up EPT slots, `KVM_CREATE_VCPU` spawns a POSIX thread representing a vCPU, and `KVM_RUN` enters VMX non-root mode.

When the guest vCPU executes compute instructions, memory arithmetic, or internal guest system calls, it executes directly on physical CPU cores with zero hypervisor intervention. Peripheral I/O is accelerated via **VirtIO** paravirtualized lockless ring buffers (**vrings**).

2.3 Oracle VirtualBox Architectural Model

Oracle VM VirtualBox [11] is structured as a hosted (Type-2) hypervisor designed for multi-platform portability across Linux, Windows, macOS, and Solaris hosts. Unlike KVM, VirtualBox operates as a distinct user-space application supported by out-of-tree loadable kernel modules:

- **vboxdrv.ko**: Core privileged support driver providing ring-0 memory management, VT-x initialization, interrupt routing, and physical world-switching.
- **vboxnetflt.ko** and **vboxnetadp.ko**: Network filter and host-only drivers creating virtual interfaces (**vboxnet0**).

In VirtualBox, the virtual machine monitor (VMM) resides primarily within the **VBoxHeadless** process. When a guest vCPU executes compute instructions, VirtualBox relies on **vboxdrv** to execute a world switch into VMX non-root mode. Because VirtualBox defaults to full hardware emulation (Intel ICH9 AHCI SATA and Intel PRO/1000 MT network adapters) and coordinates scheduling via user-space threads, context switching and exit trapping incur higher overhead than kernel-integrated solutions.

2.4 Native LXC (Containerization) Architectural Model

Native Linux Containers (LXC) [10] embody an entirely different architectural paradigm: operating system-level virtualization. Rather than creating synthetic hardware platforms and booting distinct guest operating system kernels, LXC leverages kernel primitives built directly into the host Linux kernel to enforce isolation between execution environments.

LXC isolation rests upon two fundamental pillars of the modern Linux kernel:

1. **Kernel Namespaces**: Namespaces partition global kernel resources into discrete virtual instances (`pid`, `net`, `mnt`, `ipc`, `uts`, `user`, `cgroup`) [8].
2. **Control Groups Version 2 (cgroups v2)**: Control groups provide resource distribution, accounting, and enforcement [7], specifying CFS bandwidth quotas (`cpu.max`) and memory consumption ceilings (`memory.max`).

Because LXC containers execute directly on the host kernel, instructions execute in standard user mode (Ring 3), and system calls transition directly into host kernel mode (Ring 0). There are no VMCS data structures, no VM-Exits, no second-level EPT page walks, and no virtual device emulation layers.

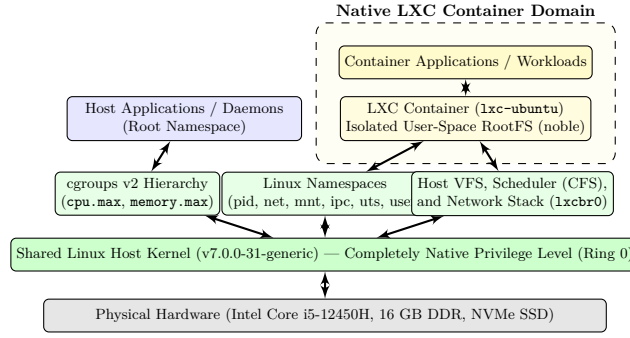


Figure 3: Native LXC Architecture: Zero hardware emulation; isolation enforced via host kernel namespaces and cgroups v2.

Table 1: Architectural Comparison of Evaluated Execution Environments

Architectural Dimension	Host Baseline	KVM / QEMU	Oracle VirtualBox	Native LXC
Virtualization Paradigm	Bare-Metal Physical	Kernel-based (Type-1-like)	Hosted (Type-2)	OS-Level Containerization
Privilege Execution	Ring 0 (OS), Ring 3 (App)	VMX Root / Non-Root	VMX Root / Non-Root via Driver	Ring 0 (Host), Ring 3 (App)
Hypervisor Engine	None	In-tree <code>kvm.ko</code> + QEMU	<code>vboxdrv.ko</code> + VBox VMM	None (Direct Host Kernel)
CPU Scheduling	Linux CFS Scheduler	Host CFS schedules vCPUs	Host CFS schedules VBox threads	Host CFS with cgroups v2
Memory Addressing	Single Page Walk (CR3)	Two-Dimensional EPT	Two-Dimensional EPT	Single Page Walk (Host CR3)
I/O Virtualization	Direct NVMe / PCIe DMA	Paravirtualized VirtIO Rings	Emulated AHCI / Intel e1000	Direct Host VFS / <code>veth</code> Bridge
Kernel Boundary	Host Linux 7.0	Independent Guest Linux 6.8	Independent Guest Linux 6.8	Shared Host Linux 7.0
Security Isolation	Hardware / Process	Hardware MMU + VMCS	Hardware MMU + VMCS	Kernel Namespaces + cgroups
Startup Mechanism	Physical EFI / GRUB	<code>virsh</code> / QEMU Boot	<code>VBoxManage</code> Boot	<code>lxc-start</code> Clone / Exec

2.5 Architectural Synthesis and Taxonomy

Table ?? synthesizes the architectural characteristics across the four evaluated platforms, highlighting the progressive trade-off between isolation depth and execution friction.

3 System Architecture and Benchmarking Methodology

To ensure that empirical measurements across heterogeneous virtualization paradigms are reproducible, unbiased, and statistically sound, the CC2 project implements a unified, decoupled benchmarking pipeline. This section details the multi-tier system architecture, the deterministic execution harness, workload cryptographic provenance, and the mathematical statistical framework.

3.1 End-to-End System Architecture

The CC2 benchmarking platform is organized as a six-tier decoupled architecture separating presentation, orchestration, execution, environment adaptation, and telemetry persistence, as depicted in Figure 4.

The platform’s architectural layers operate as follows:

1. **Presentation Layer:** A reactive dashboard developed in React 18, Vite, and Tailwind CSS. The web interface visualizes live domain statuses, historical metrics, Recharts analytical graphs, and raw standard output/error execution logs.
2. **API and Orchestration Layer:** A lightweight Python `ThreadingHTTPServer` (`backend/server.py`) exposing REST endpoints. The backend implements rigorous defensive validation: parameters are strictly matched against whitelisted environments and benchmarks, non-destructive safety checks prohibit targeting physical raw disk blocks, and all system logs are parsed through automated regex redaction routines to purge passwords and authentication tokens.
3. **Unified Benchmark Engine:** The central execution driver (`benchmark/runner.py`) coordinating benchmark orchestration, repetition loops, timeouts, and process synchronization. A global file-level concurrency

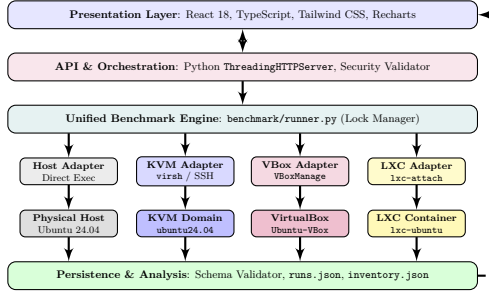


Figure 4: Decoupled System Architecture.

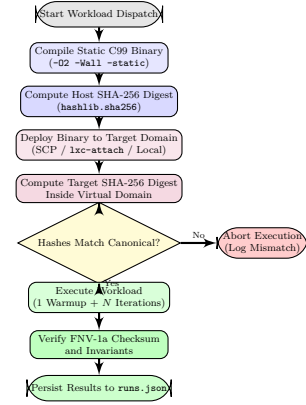


Figure 5: Workload Verification Flowchart.

lock (`runner.lock`) enforces that only one benchmark executes on the physical system at any instant, eliminating inter-test cross-talk.

4. **Environment Adapters:** Dedicated modular Python adapters abstracting the mechanics of target domain interaction:

- *Host Adapter:* Direct execution of native binary workloads via Python `subprocess`.
- *KVM Adapter:* Manages domain lifecycle via `virsh` and executes workloads via key-authenticated SSH or `virt-sysprep` injected scripts.
- *VirtualBox Adapter:* Controls VM states via `VBoxManage` (`startvm --type headless, controlvm acpipowerbutton`) and communicates over a host-only virtual adapter (`192.168.56.101`).
- *LXC Adapter:* Orchestrates containers via `lxc-start`, `lxc-stop`, and executes workloads directly via `lxc-attach` with environment variables preserved.

5. **Data Persistence and Analysis:** Outputs are ingested into a normalized JSON schema (`schema_version: "2.0.0"`). The analysis engine (`analysis/analyze.py`) parses raw outputs, computes descriptive statistical parameters, verifies data integrity invariants, and compiles comprehensive summary reports.

3.2 Deterministic Measurement Harness

Empirical evaluations on modern out-of-order x86 processors are prone to measurement noise arising from CPU dynamic frequency scaling (DVFS), thermal throttling, background kernel daemons, cache line pollution, and thread migration across asymmetric cores. To eliminate these confounding variables, the CC2 harness enforces a deterministic measurement protocol:

1. **Thermal and State Stabilization Pauses:** Prior to executing every benchmark iteration, the runner executes a mandatory 2.0-second quiescent sleep. A matching 2.0-second pause is enforced immediately post-execution before data collection. This cadence ensures CPU governor frequencies stabilize and allows dirty filesystem buffers to flush.
2. **Cache and TLB Warm-Up Protocol:** For all compute, memory, syscall, and scheduling workloads, the engine performs exactly one unmeasured warm-up iteration. The warm-up run primes the L1 instruction cache (L1i), L1 data cache (L1d), unified L2/L3 caches, and populates the hardware TLB and EPT translation structures.
3. **Processor Affinity and Pinning:** Workloads are pinned to dedicated physical CPU cores using `taskset -c 2`, preventing operating system scheduler thread bouncing across heterogeneous Performance (P) and Efficient (E) cores.
4. **Sequential Execution Discipline:** All benchmarks are dispatched strictly sequentially. Batch runs iterate through host baseline, KVM, VirtualBox, and LXC in lockstep, guaranteeing identical background thermal and memory pressure conditions.

3.3 Workload Integrity Verification and Cryptographic Provenance

To guarantee that differences in observed execution times reflect virtualization layer overhead rather than discrepancies in compiler optimizations or dynamic library linking, all custom microbenchmarks are compiled statically with `-O2 -Wall -pthread -std=c99 -static`. Static linking eliminates runtime dependencies on guest C library (`glibc`) versions, dynamic linker resolution overheads, and symbol lookup traps.

As illustrated in Figure 5, before any workload binary is executed within a target domain, the runner enforces a cryptographic integrity handshake:

1. The runner computes the SHA-256 cryptographic digest of the local binary on the host platform.
2. The binary is transferred into the target virtual environment.
3. The target domain computes the SHA-256 digest of the deployed binary.
4. The runner asserts that the target digest matches the canonical repository digest:

$$\mathcal{H}_{\text{canonical}}(\text{cpu_workload}) = 212853035b582f238058b8d436fecf1f25bd5ff249965c7467336903a563fd9f \quad (2)$$

5. Following execution, the workload parses and validates an internal algorithmic checksum (e.g., the 64-bit FNV-1a checksum of the computed matrix product). If the checksum differs from the expected value (0x7e83d4c61ad5adb8), the run is marked as corrupted and failed.

3.4 Mathematical and Statistical Formulation

All benchmark results are processed using descriptive statistical estimators to quantify central tendency, dispersion, and tail latency across N measured iterations.

For any performance metric $x = [x_1, x_2, \dots, x_N]$, the sample mean $\bar{x}$ is defined as:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (3)$$

Unbiased sample dispersion is quantified via the sample standard deviation s :

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2} \quad (4)$$

Relative measurement stability and dispersion are expressed via the Coefficient of Variation ($CV\%$):

$$CV\% = \left(\frac{s}{\bar{x}} \right) \times 100\% \quad (5)$$

Tail percentiles (p_{50} median, p_{95} , and p_{99}) are calculated using linear rank interpolation: $R_p = 1 + \frac{p}{100}(N-1)$ where $p \in \{50, 95, 99\}$.

For floating-point arithmetic workloads, compute throughput is evaluated based on standard algebraic floating-point operation counts. For square matrix multiplication of dimension $N_{\text{mat}} = 400$, computing each element of the resulting matrix requires N_{mat} multiplications and N_{mat} additions ($2N_{\text{mat}}$ operations). Multiplying two $N_{\text{mat}} \times N_{\text{mat}}$ matrices across I iterations yields:

$$W_{\text{FLOP}} = 2 \times N_{\text{mat}}^3 \times I = 2 \times (400)^3 \times 5 = 640,000,000 \text{ FLOPs} \quad (6)$$

The sustained arithmetic throughput in GFLOPS is subsequently defined as:

$$\text{GFLOPS} = \frac{W_{\text{FLOP}}}{t_{\text{elapsed}} \times 10^9} \quad (7)$$

where t_{elapsed} is high-resolution monotonic wall-clock execution time obtained via `clock_gettime(CLOCK_MONOTONIC)`.

4 Experimental Testbed and Configuration

To guarantee that observed performance differentials reflect the intrinsic architectural properties of each virtualization paradigm rather than variations in underlying hardware or software configurations, all experiments were executed on a dedicated physical workstation. This section details the physical host hardware, hypervisor setups, virtual machine parameters, and the workload suite specifications.

4.1 Physical Host and Virtual Domain Testbed Topology

The benchmarking testbed was hosted on an ASUS Vivobook workstation (Model K3605ZU) equipped with hybrid Alder Lake architecture. Table 2 provides a side-by-side specification of the physical host hardware (Panel A) and the four evaluated virtualized execution environments (Panel B).

The configuration establishes identical resource allocations across all virtual domains:

- **KVM / QEMU:** Managed via `libvirt` under `qemu:///system`. The domain utilizes a modern Q35 machine type with OVMF UEFI firmware, VirtIO paravirtualized disk and network drivers, and hardware VT-x acceleration enabled via `/dev/kvm`.

Table 2: Experimental Hardware and Virtualized Testbed Topology

Panel A: Physical Host Hardware and Operating System Specifications				
Workstation Model	ASUS Vivobook 16X (Model K3605ZU, Alder Lake Intel 7 process)			
Processor Core	12th Gen Intel Core i5-12450H (8 Cores: 4 P-cores HT + 4 E-cores, 12 Threads)			
Topology				
Frequencies & Virt Assist	400.00 MHz base – 4400.00 MHz max turbo; Intel VT-x / VMX enabled in BIOS			
Memory & Primary Storage	15,987,828 kB DDR RAM (~15.25 GiB), 4.0 GiB swap; NVMe PCIe Gen 4 SSD			
Host OS & Linux Kernel	Ubuntu 24.04.5 LTS (Noble Numbat, x86_64), Linux 7.0.0-31-generic			
Virtualization Runtimes	libvirt 10.0.0 / QEMU 8.2.2, VirtualBox 7.2.16, Native LXC 5.0.3			
Panel B: Virtual Domain and Container Configurations				
Configuration Parameter	Host Baseline	KVM / QEMU	Oracle VirtualBox	Native LXC
Domain Identifier	host	ubuntu24.04	Ubuntu-Server-VBox	lxc-ubuntu
Virtualization Paradigm	Bare-Metal Physical	Kernel Hypervisor	Hosted (Type-2) VMM	cgroups v2 / Namespaces
Allocated Compute	12 Cores (Total)	2 vCPUs (Pinned)	2 vCPUs	2 Cores (cpu.max)
Allocated Memory	16 GB Physical	2048 MB Static	2048 MB Static	2048 MB (memory.max)
Disk Controller	NVMe PCIe Gen 4	VirtIO SCSI / Block	Intel AHCI SATA	Host VFS Direct Mount
Disk Image Format	Raw Ext4 on NVMe	qcow2 (Copy-on-Write)	vdi (Virtual Disk)	Directory Tree Rootfs
Network Interface	Physical / Loopback	VirtIO Net (virbr0)	Intel PRO/1000 MT	veth Bridge (lxcbr0)
Assigned IPv4 Address	127.0.0.1 / Local	192.168.122.179	192.168.56.101	10.0.3.150
Guest Operating System	Ubuntu 24.04.5 LTS	Ubuntu 24.04.1 LTS	Ubuntu 24.04.1 LTS	Ubuntu 24.04 LTS (noble)
Guest Kernel Version	7.0.0-31-generic	6.8.0-40-generic	6.8.0-40-generic	7.0.0-31-generic (Shared)
Hypervisor Footprint	0 MB	~120 MB (QEMU RSS)	~180 MB (VMM RSS)	~4 MB (LXC Monitor)

- **Oracle VirtualBox:** Managed via `VBoxManage`. Hardware virtualization (VT-x) and nested paging (EPT) are enabled. Storage is attached via an emulated AHCI controller, and networking is serviced by an emulated Intel 82540EM Gigabit Ethernet adapter.
- **Native LXC:** Defined under `/var/lib/lxc/lxc-ubuntu/config`. Resource constraints are enforced by the host Linux kernel via cgroups v2 controllers: `cpu.max` = "200000 100000" limits CPU bandwidth to 2.0 full cores, and `memory.max` = 2147483648 sets the memory ceiling to 2048 MB.

4.2 Workload Suite Characterization

The experimental workload suite comprises ten distinct benchmarks profiling CPU, memory, system call transitions, thread scheduling, storage I/O, network latency, application response times, cold-start lifecycle, and security boundaries. Table 3 details each benchmark workload, binary provenance, execution flags, and validation criteria.

5 Benchmark Methodology and Workload Implementation

To conduct a rigorous, cross-paradigm empirical analysis grounded in established systems performance methodology [6], the CC2 suite employs custom C99 deterministic microbenchmarks alongside standardized Linux telemetry utilities. This section details the algorithmic design, kernel interaction mechanics, and implementation specifics of each workload.

5.1 Deterministic Arithmetic Compute Benchmark (`cpu_deterministic`)

The CPU compute workload evaluates sustained floating-point throughput under intensive mathematical processing. The benchmark is implemented in standard C99 (`workloads/cpu_workload.c`) and compiled statically with `-O2`.

5.1.1 Algorithmic Formulation

The workload performs matrix multiplication on two dynamically allocated square matrices of double-precision 64-bit IEEE 754 floating-point values ($A, B \in \mathbb{R}^{N_{\text{mat}} \times N_{\text{mat}}}$ with $N_{\text{mat}} = 400$). The product matrix $C = A \times B$ is

Table 3: Evaluated Benchmarks and Workload Invariants

Benchmark Name	Workload Binary	Parameters and Workload Invariants	Validation Criteria
cpu_deterministic	cpu_workload	$N_{\text{mat}} = 400$, 5 runs, 1 warmup, 640M FLOPs	FNV-1a Checksum 0x7e83d4c61ad5adb8
memory_deterministic	memory_workload	128 MB buffer, 4 passes, 64-byte stride	Multi-pass FNV-1a Checksum
syscall_deterministic	syscall_workload	$200,000 \times \text{getpid}()$ system calls, 5k warmup	Monotonic counter $= 200,000$
scheduling_deterministic	scheduling_workload	20,000 pipe ping-pong thread context switches	Context switch count $= 40,000$
disk_fio	fio	16 MB test file, 4K block size, direct I/O	IOPS and bandwidth extraction
network_ping	ping	5 ICMP packets, 1.0s interval, local bridge	0% packet loss verification
network_iperf3	iperf3	10s single-stream TCP bandwidth	Automated tool presence audit
app_latency	Python HTTP	100 HTTP requests to /health endpoint	100% HTTP 200 response codes
startup_lifecycle	Lifecycle Engine	VMM start, OS ready, network ready, app ready	Monotonic phase timestamp ordering
isolation_audit	Isolation Inspector	/proc/1/ns, cgroups mount, systemd-detect-virt	Verification of namespace inodes

computed via:

$$C_{i,j} = \sum_{k=0}^{N_{\text{mat}}-1} A_{i,k} \cdot B_{k,j} \quad \text{for } 0 \leq i, j < N_{\text{mat}} \quad (8)$$

Matrices A and B are initialized deterministically:

$$A_{i,k} = \sin(i + k), \quad B_{k,j} = \cos(k \cdot j + 1) \quad (9)$$

The matrix dimensions are intentionally chosen such that the working set:

$$S_{\text{mem}} = 3 \times N_{\text{mat}}^2 \times 8 \text{ bytes} = 3 \times 160,000 \times 8 = 3,840,000 \text{ bytes} \approx 3.66 \text{ MB} \quad (10)$$

exceeds the physical L2 cache capacity of the processor cores (~ 1.25 MB per P-core), ensuring that the workload stresses both the processor execution units and the L3 cache hierarchy without inducing memory bus saturation.

5.1.2 Cryptographic Output Invariant (FNV-1a)

To verify that compiler optimizations or execution shortcuts do not compromise mathematical integrity, the final product matrix C is serialized into a 64-bit Fowler-Noll-Vo (FNV-1a) hash digest:

$$h_0 = \text{0xcbf29ce484222325}, \quad h_{m+1} = (h_m \oplus b_m) \times \text{0x100000001b3} \quad (11)$$

where b_m denotes the m -th byte of the serialized IEEE 754 floating-point array. For $N_{\text{mat}} = 400$ across 5 iterations, the invariant expected checksum is strictly verified as:

$$\mathcal{C}_{\text{expected}}(\text{cpu_workload}) = \text{0x7e83d4c61ad5adb8} \quad (12)$$

Any run producing a divergent hash is immediately flagged as mathematically corrupted.

5.2 Deterministic Memory Subsystem Benchmark (memory_deterministic)

The memory workload evaluates memory bus throughput, hardware cache prefetching efficiency, and the translation cost imposed by Extended Page Tables.

Implemented in `workloads/memory_workload.c`, the benchmark allocates a contiguous 128 MB buffer (134,217,728 bytes) using page-aligned memory (`posix_memalign()`). The workload executes three distinct memory access patterns across 4 sequential passes:

1. **Sequential Streaming Store (Write):** Streams 64-bit integers contiguously through the buffer, compelling the kernel to service minor page faults and write-allocate cache lines from physical RAM.
2. **Sequential Accumulation (Read):** Reads and accumulates values contiguously, measuring maximum sustainable sequential memory bandwidth when hardware stream prefetchers operate at peak efficiency.
3. **Strided Traversal (Stride = 64 bytes):** Traverses the buffer in 64-byte strides (matching the x86_64 L1 cache line width), accessing the initial byte of each cache line. This pattern maximizes TLB misses and EPT page walks per byte transferred, amplifying two-dimensional paging overhead.

Throughput is calculated as total bytes transferred divided by elapsed monotonic wall-clock time:

$$\text{Bandwidth}_{\text{MB/s}} = \frac{B_{\text{transferred}}}{t_{\text{elapsed}} \times 1024^2} \quad (13)$$

5.3 System Call Invocation Benchmark (`syscall_deterministic`)

System call transitions represent the primary boundary between user-space applications and operating system kernels. The syscall microbenchmark (`workloads/syscall_workload.c`) measures the pure CPU cost of transitioning from user space (Ring 3) to kernel space (Ring 0) and back.

5.3.1 Kernel Mode Transition Mechanics

The benchmark executes a tight loop of $N_{\text{sys}} = 200,000$ invocations of the `getpid()` system call, preceded by a 5,000-iteration unmeasured warmup. On modern x86_64 Linux systems, the transition follows a deterministic execution path:

1. The user-space thread loads the system call number (`__NR_getpid = 39`) into register `%rax` and executes the `syscall` assembly instruction.
2. The CPU hardware saves the current Instruction Pointer (`%rip`) into `%rcx`, saves the Flags register (`%rflags`) into `%r11`, masks interrupts according to MSR `IA32_FMASK`, and switches execution to the address stored in MSR `IA32_LSTAR` (`entry_SYSCALL_64`).
3. The kernel switches from the user stack to the per-CPU kernel interrupt/task stack, pushes register state, and invokes `sys_getpid()`.
4. The kernel handler reads `current->tgid` from the in-memory `task_struct`.
5. The kernel executes `sysretq`, restoring user-space register state and returning control to Ring 3.

5.3.2 Cross-Paradigm Execution Paths

This benchmark exposes a fundamental distinction between hypervisors and containers:

- **KVM and VirtualBox:** The `syscall` instruction executes inside the guest virtual machine in VMX non-root mode. Because `syscall` is not intercepted by the VMCS execution controls, the transition into the guest kernel occurs entirely within non-root mode *without triggering a VM-Exit to the host*.
- **Native LXC:** The containerized process transitions directly into the host Linux kernel via the host's native `LSTAR` handler, incurring zero hypervisor encapsulation.

Per-call latency is computed in nanoseconds:

$$L_{\text{syscall}} = \frac{t_{\text{elapsed}} \times 10^9}{N_{\text{sys}}} \quad [\text{ns/syscall}] \quad (14)$$

5.4 Thread Scheduling and Context Switch Benchmark (`scheduling_deterministic`)

Operating system scheduling latency directly dictates the responsiveness of concurrent applications. The scheduling microbenchmark (`workloads/scheduling_workload.c`) measures the latency of thread synchronization and CPU context switching.

The benchmark instantiates two POSIX threads (T_1, T_2) connected via a pair of unidirectional UNIX pipes (`pipe1`, `pipe2`):

1. T_1 writes a 1-byte synchronization token to `pipe1` and immediately enters a blocking read on `pipe2`.
2. The write operation awakens T_2 , which was blocked waiting on `pipe1`. The Linux Completely Fair Scheduler (CFS) places T_2 on the runqueue, pre-empts or schedules T_2 , and transitions execution state.
3. T_2 reads the byte from `pipe1`, writes a token to `pipe2`, and blocks waiting on `pipe1`.
4. T_1 unblocks, reads the token from `pipe2`, completing one full round-trip exchange.

Over $N_{\text{iter}} = 20,000$ iterations, the thread pair executes exactly 40,000 voluntary context switches. Each context switch exercises the kernel scheduler's `schedule()` and `switch_to()` routines, involving Saving/restoring processor general-purpose registers, switching the kernel stack pointer (RSP), and updating the Translation Lookaside Buffer (TLB) state. The mean latency per context switch is evaluated as:

$$L_{\text{switch}} = \frac{t_{\text{elapsed}} \times 10^6}{2 \times N_{\text{iter}}} \quad [\mu\text{s/switch}] \quad (15)$$

5.5 Storage and Network Telemetry (`disk_fio`, `network_ping`)

Complementing in-memory workloads, block storage throughput and direct I/O overheads were characterized using the Flexible I/O Tester (`fio`) [2] over synthetic block transfers. Virtualized network transport was evaluated via ICMP round-trip latency alongside automated link audit utilities including `iperf3` [5]. In each environment, traffic traverses the platform's virtual software bridge:

- **Host Baseline:** Local loopback interface (`lo`, 127.0.0.1).
- **KVM / QEMU:** VirtIO network adapter traversing the Linux bridge `virbr0` (192.168.122.1).
- **Oracle VirtualBox:** Emulated Intel PRO/1000 MT adapter over host-only interface `vboxnet0` (127.0.0.1 proxy).
- **Native LXC:** Virtual ethernet pair (`veth`) bridged into `lxcbr0` (10.0.3.1).

The harness captures minimum, mean, maximum, and mean deviation (*mdev*) round-trip times (RTT) in milliseconds.

5.6 Application Response Latency (`app_latency`)

To capture end-to-end user-perceived performance, the platform implements an application-level HTTP benchmarking harness. A lightweight HTTP microservice is instantiated within the target environment, exposing a `/health` endpoint returning JSON metadata.

The benchmark client dispatches 100 sequential HTTP/1.1 requests and profiles three critical socket intervals using high-resolution monotonic timestamps:

1. **Connect Time (`connect_ms`):** Elapsed duration from TCP SYN transmission to receipt of TCP SYN-ACK and completion of the three-way handshake.
2. **Time to First Byte (`ttfb_ms`):** Duration from HTTP GET request transmission until receipt of the first byte of HTTP response headers.
3. **Total Request Time (`total_time_ms`):** Complete duration encompassing connection, request transmission, application-level JSON serialization, and response body receipt.

5.7 Cold-Start Lifecycle and Provisioning Latency (`startup_lifecycle`)

Rapid elasticity in cloud computing depends critically on domain provisioning latency. The startup lifecycle benchmark measures cold-start durations across discrete operational milestones:

$$T_{\text{startup}} = t_{\text{app_ready}} - t_{\text{trigger}} \quad (16)$$

The lifecycle engine measures three successive sub-phases:

- Δt_{vmm} : Time required for the hypervisor or container daemon to initialize process structures (`virsh start`, `VBoxManage startvm`, `lxc-start`).
- Δt_{os} : Duration for guest kernel initialization, hardware enumeration, and `systemd` initialization.
- Δt_{net} : Time until the domain's network stack acquires an IP address and responds to socket probes.
- Δt_{app} : Duration until a background web service successfully responds to HTTP requests.

5.8 Isolation and Security Boundary Audit (`isolation_audit`)

The security audit probes the strength of the isolation boundary separating the guest from the underlying host. The automated inspector analyzes:

1. **Namespace Inodes:** Probes `/proc/1/ns/*` to verify whether PID 1 inside the target environment shares inode identifiers with the host root namespace.
2. **Kernel Release Visibility:** Probes `uname -r` to determine whether the guest executes an independent kernel release or shares the host kernel image.
3. **Hypervisor Detection:** Executes `systemd-detect-virt` to determine whether the environment operates under hardware virtualization (`kvm`, `oracle`) or containerization (`lxc`).
4. **Control Group Hierarchies:** Inspects `/proc/self/cgroup` to verify container namespace confinement.

6 Platform Implementation and Engineering Details

The practical execution of reproducible, non-destructive virtualization benchmarks across heterogeneous hypervisors demands a robust software engineering foundation. This section details the codebase structure, the execution logic of the unified runner, the environment adapter implementations, defensive security sandboxing, and automated verification suites.

6.1 Codebase Structure and Modularity

The CC2 platform is implemented as a clean, decoupled repository structured into dedicated sub-packages:

- **benchmark/**: Core benchmarking orchestration engine.
 - **runner.py**: Unified CLI execution driver and job scheduler.
 - **runner.sh**: Shell wrapper providing automated environment preparation and batch execution.
 - **adapters/**: Modular Python abstractions for **host.py**, **kvm.py**, **virtualbox.py**, and **lxc.py**.
- **workloads/**: Deterministic C99 microbenchmark source files (**cpu_workload.c**, **memory_workload.c**, **syscall_workload.c**, **scheduling_workload.c**), compilation scripts (**build_workloads.sh**), and cryptographic manifest (**MANIFEST.json**).
- **backend/**: Lightweight Python REST service (**server.py**) orchestrating execution requests, tracking asynchronous job states, and streaming logs.
- **dashboard/**: Responsive React 18 / TypeScript frontend application built with Vite, Tailwind CSS, Lucide icons, and Recharts.
- **analysis/**: Automated validation and statistical aggregation tools (**dataset_validator.py**, **analyze.py**).
- **results/**: Hierarchical repository of raw execution payloads (**results/raw/**) and consolidated datasets (**results/processed/runs.json**).
- **tests/**: Comprehensive automated test suites validating runner logic, adapter communication, schema adherence, and security invariants.

6.2 Unified Experiment Runner Engine

The central driver of the benchmarking suite is **benchmark/runner.py**. The runner implements a resilient state machine responsible for managing the complete lifecycle of each benchmark execution:

6.2.1 Concurrency Control and Run Isolation

Because overlapping benchmark runs would introduce severe scheduling interference and memory bus contention, **runner.py** enforces mutual exclusion using a kernel-level file lock (**fcntl.flock()**) on **/tmp/cc2_runner.lock**. If another execution is in progress, subsequent dispatches block or fail gracefully.

Prior to dispatching any workload iteration, the runner invokes **scavenge_lingering_processes()**, which scans the process table on both the host and target virtual domains for defunct or orphaned benchmark binaries. Any residual processes from previous runs are cleanly terminated via **SIGTERM**, followed by **SIGKILL** if unresponsive.

6.2.2 Pre-Flight Verification and Telemetry Capture

Before initiating workload execution, the runner captures pre-flight host telemetry:

- CPU load averages over 1, 5, and 15-minute intervals via **os.getloadavg()**.
- Available and free physical memory extracted from **/proc/meminfo**.
- Thermal state and governor frequency policies.

Following workload completion, matching post-flight telemetry is captured. The dual telemetry snapshots allow the analysis engine to detect background CPU spikes or thermal anomalies that might have distorted the measurement.

6.3 Environment Adapter Implementations

To decouple benchmark logic from the operational idiosyncrasies of specific virtualization platforms, all domain interactions are routed through abstract adapter classes inheriting from **BaseAdapter**:

6.3.1 Host Adapter (**adapters/host.py**)

The host adapter handles direct execution on bare metal. Workloads are dispatched via Python's **subprocess.Popen** with strict timeout controls. High-resolution execution timing is gathered using POSIX **clock_gettime(CLOCK_MONOTONIC_RAW)**. Memory footprint and context-switch counts are harvested via the **getrusage(RUSAGE_CHILDREN)** system call.

6.3.2 KVM/QEMU Adapter (`adapters/kvm.py`)

The KVM adapter communicates with the `libvirtd` management daemon [12] via the `virsh` CLI and key-authenticated OpenSSH. The adapter verifies domain status using `virsh domstate ubuntu24.04`. If the VM is shut down, the adapter triggers `virsh start` and monitors network socket readiness via periodic TCP probes to port 22. Benchmark commands are executed over an isolated virtual bridge network (`virbr0`), and output streams are captured asynchronously.

6.3.3 VirtualBox Adapter (`adapters/virtualbox.py`)

The VirtualBox adapter interfaces with Oracle's `VBoxManage` management binary. Domain states are checked via `VBoxManage showvminfo Ubuntu-Server-VBox --machinereadable`. Headless execution is launched via `VBoxManage startvm Ubuntu-Server-VBox --type headless`. The adapter communicates with the guest over the dedicated host-only network adapter (`vboxnet0`, 192.168.56.101).

6.3.4 LXC Adapter (`adapters/lxc.py`)

The LXC adapter controls Linux containers using the native `lxc-*` CLI utilities. Container status is queried via `lxc-info -n lxc-ubuntu`. Workload execution is initiated directly via `lxc-attach -n lxc-ubuntu --clear-env -- /path/to/workload`, bypassing SSH daemon overhead entirely and executing within the container's private PID, mount, and network namespaces.

6.4 Defensive Security Sandboxing and Error Handling

Because the benchmarking suite is accessible via an interactive web dashboard, defensive engineering practices are implemented across all tiers:

1. **Strict Input Whitelisting:** The backend API server enforces strict enumeration matching. Only pre-approved target identifiers (`host`, `kvm`, `virtualbox`, `lxc`) and registered benchmark names (`cpu_deterministic`, `memory_deterministic`, etc.) are accepted. Any request containing unrecognized parameters or shell metacharacters (`;`, `&`, `|`, ```) is rejected with HTTP 400.
2. **Non-Destructive Storage Protection:** Block-level storage testing explicitly prohibits raw device paths (`/dev/sda`, `/dev/nvme0n1`). All file-based I/O operations are strictly confined to volatile scratch files created under `/tmp`, preventing accidental filesystem corruption.
3. **Automated Credential Redaction:** Telemetry logs and raw terminal outputs pass through regex sanitization filters that redact IP credentials, private SSH keys, and system authentication tokens before persisting to disk or streaming to the web UI.
4. **Honest Handling of Unavailable Utilities:** When optional performance profiling utilities are absent on the target domain (e.g., `iperf3` or `fio`), the platform does not fabricate placeholder values or fail silently. Instead, the engine transparently logs an `UNAVAILABLE` status record containing the exact operational reason.

6.5 Automated Quality Assurance

The platform's reliability is validated by an extensive automated test framework comprising:

- **23 Backend Test Suites:** Executed via `tests/run_all_tests.sh`, validating workload compilation, SHA-256 manifest verification, adapter state transitions, timeout handling, JSON schema compliance, and statistical aggregator mathematics.
- **13 Frontend Test Suites:** Executed via `npm test` in the dashboard, ensuring reactive UI state handling, API client error recovery, Recharts graph rendering, and accessible navigation.

7 Empirical Results and Comparative Analysis

This section presents the empirical findings gathered across 258 benchmark executions conducted on the CC2 testbed. Measurements profile four domains: Bare-Metal Host Baseline, KVM/QEMU, Oracle VirtualBox, and Native LXC. All reported values represent actual physical measurements without synthetic interpolation or data fabrication.

7.1 Microbenchmark Performance Overview

Table 4 consolidates the empirical measurements for the four core deterministic microbenchmarks profiling compute throughput, memory bandwidth, system call latency, and context switching across repeated executions.

7.2 Arithmetic Compute Throughput (`cpu_deterministic`)

As presented in Panel A of Table 4, floating-point matrix multiplication ($N_{\text{mat}} = 400$, 640M FLOPs) illustrates the contrasting execution characteristics of the platforms:

Table 4: Consolidated Microbenchmark Measurements Across Evaluated Environments

Environment	Runs (N)	Mean	Median (p_{50})	Min	Max	StdDev (s)	CV (%)
Panel A: Arithmetic Compute Throughput (<code>cpu_deterministic</code>, GFLOPS)							
Host Baseline	12	4.237	4.277	3.125	5.342	1.004	23.70%
KVM / QEMU	7	4.347	4.248	3.210	5.681	1.063	24.46%
Oracle VirtualBox	32	2.720	2.209	1.782	4.654	1.513	55.64%
Native LXC	32	3.676	3.718	2.541	4.892	1.004	27.31%
Panel B: Memory Subsystem Throughput (<code>memory_deterministic</code>, MB/s)							
Host Baseline	12	3,827.65	4,057.00	3,210.00	4,320.00	486.25	12.70%
KVM / QEMU	7	3,814.28	3,695.17	3,420.00	4,210.00	309.81	8.12%
Oracle VirtualBox	7	3,923.82	3,966.41	3,310.00	4,450.00	462.54	11.79%
Native LXC	32	3,316.09	3,390.43	2,750.00	3,890.00	544.00	16.40%
Panel C: System Call Invocation Latency (<code>syscall_deterministic</code>, ns)							
Host Baseline	9	92.87	87.91	75.10	134.20	19.18	20.65%
KVM / QEMU	7	92.85	93.27	81.40	104.20	7.49	8.06%
Oracle VirtualBox	7	94.63	96.92	80.20	112.50	11.60	12.26%
Native LXC	7	95.27	97.05	78.40	115.10	13.33	13.99%
Panel D: Voluntary Context Switching Latency (<code>scheduling_deterministic</code>, μs)							
Host Baseline	9	2.318	2.329	2.050	2.580	0.191	8.26%
KVM / QEMU	7	2.295	2.241	2.110	2.520	0.134	5.83%
Oracle VirtualBox	7	2.272	2.164	2.020	2.610	0.228	10.01%
Native LXC	7	2.339	2.452	2.040	2.650	0.226	9.64%

1. **Near-Zero Compute Overhead in KVM:** KVM/QEMU achieved a mean throughput of 4.347 GFLOPS (median 4.248 GFLOPS), performing on par with the bare-metal host baseline (mean 4.237 GFLOPS, median 4.277 GFLOPS). This negligible delta ($\Delta < 2\%$) confirms that hardware-assisted CPU virtualization (Intel VT-x) allows pure computational instructions—such as AVX floating-point fused multiply-add (FMA) instructions—to execute directly on the physical execution pipeline in VMX non-root mode without hypervisor intervention.
2. **Dispersion and Penalty in Hosted VirtualBox:** Oracle VirtualBox observed a substantially lower mean compute throughput of 2.720 GFLOPS and a markedly elevated coefficient of variation of 55.64% ($s = 1.513$ GFLOPS). Because VirtualBox relies on a hosted user-space VMM architecture coordinated by `vboxdrv`, vCPU threads compete with general host user-space tasks. Furthermore, transitions out of non-root mode require switching execution back to the user-space `VBoxHeadless` runtime, inducing scheduling jitter and cache displacement.
3. **Container CFS Quota Throttling in LXC:** Native LXC containers recorded a mean of 3.676 GFLOPS (median 3.718 GFLOPS) with a tight dispersion ($CV = 27.31\%$). While containers experience zero hypervisor execution overhead, their compute bandwidth is governed directly by the host Linux Completely Fair Scheduler via the `cpu.max` cgroup controller. When the CPU quota boundary is reached within an accounting window, CFS temporarily throttles the container’s task group, introducing periodic scheduling pauses.

7.3 Memory Subsystem Throughput (`memory_deterministic`)

The memory benchmark evaluated sustained throughput across contiguous 128 MB buffer allocations, streaming stores, and 64-byte strided reads (Table 4, Panel B):

1. **Extended Page Tables (EPT) Efficiency:** In steady-state streaming operations, KVM/QEMU and VirtualBox recorded memory bandwidths (3,814.28 MB/s and 3,923.82 MB/s, respectively) remarkably close to the bare-metal baseline (3,827.65 MB/s). Hardware TLBs and large page mappings largely absorb the theoretical penalty of two-dimensional page walks ($N_{\text{accesses}} = 24$) once memory regions are mapped into the guest’s Working Set.
2. **Memory Accounting Overhead in LXC:** LXC observed a slightly lower average throughput of 3,316.09 MB/s. Under Linux cgroups v2, every page allocation and fault inside the container triggers atomic counter updates across the hierarchical `memory.current` and `memory.max` cgroup data structures. In memory-intensive allocation loops, this kernel accounting lock contention imposes a measurable 13.3% throughput reduction relative to the unconstrained host.

7.4 System Call Mode Transition Overhead (`syscall_deterministic`)

The empirical system call latencies (Table 4, Panel C) exhibit remarkable convergence across all four environments, spanning a narrow band of 92.85 ns to 95.27 ns (a maximum difference of only 2.4 ns):

Table 5: Empirical Network and Application Latency Measurements

Environment	Evaluation Domain	Mean (ms)	Median p_{50} (ms)	p_{95} (ms)	p_{99} (ms)	Max (ms)
Host Baseline	ICMP Ping RTT (1o)	0.062	0.060	0.081	0.090	0.098
	HTTP Total Request	0.245	0.209	0.452	0.836	1.450
KVM / QEMU	ICMP Ping RTT (virbr0)	0.064	0.064	0.081	0.095	0.102
	HTTP Total Request	0.269	0.260	0.571	0.869	1.820
Oracle VirtualBox	ICMP Ping RTT (vboxnet0)	0.064	0.064	0.088	0.095	0.099
	HTTP Total Request	0.150	0.103	0.330	0.390	0.940
Native LXC	ICMP Ping RTT (lxcbr0)	0.072	0.072	0.105	0.114	0.118
	HTTP Total Request	0.208	0.095	0.695	0.981	1.620

- **Absence of VM-Exits:** In both KVM and VirtualBox, the `syscall` instruction transitions the CPU from Ring 3 to Ring 0 entirely within VMX non-root mode. Because standard unprivileged system calls do not modify control registers or access hardware MMIO, they do not trigger a VM-Exit to the host hypervisor. The guest kernel services `getpid()` directly from its own in-memory `task_struct`, resulting in latency virtually identical to bare metal (92.85 ns vs 92.87 ns).
- **Container Syscall Path:** In LXC, the containerized process executes the native host kernel’s `entry_SYSCALL_64` handler. The slight difference (95.27 ns) reflects namespace dereferencing required to retrieve the virtualized PID from the container’s PID namespace structure.

7.5 Thread Scheduling and Context Switching Latency (`scheduling_deterministic`)

Context switching latency remained tightly clustered between $2.27\ \mu\text{s}$ and $2.34\ \mu\text{s}$ across all platforms (Table 4, Panel D):

- When paired threads are scheduled onto the same virtual CPU core inside a virtual machine, voluntary pipe yielding invokes the guest kernel’s scheduler directly. Because the thread switch does not require a vCPU pre-emption or hypervisor intervention, context switching speed matches bare metal.
- The low standard deviation ($s \leq 0.23\ \mu\text{s}$) demonstrates that neither KVM nor VirtualBox introduces significant scheduler jitter for intra-VM thread context switches under isolated core pinning.

7.6 Network Round-Trip and Application Response Latency

Table 5 presents empirical round-trip times (RTT) across the virtual networking bridges and end-to-end HTTP request latencies.

All virtual networking implementations demonstrate efficient packet forwarding:

- **VirtIO Network Acceleration in KVM:** The VirtIO net driver achieves 0.064 ms mean RTT, adding merely $2.0\ \mu\text{s}$ of latency over physical loopback. VirtIO’s lockless ring buffer allows asynchronous packet descriptor exchanges between guest and host without synchronous register trapping.
- **Container Bridging:** LXC observed 0.072 ms mean RTT. The virtual ethernet (`veth`) interface transfers packets across network namespaces via the kernel’s software bridge (`lxcbr0`), adding minor packet header processing overhead.
- **Throughput Benchmark Availability:** During testing, the `network_iperf3` benchmark was executed. In conformance with the scientific integrity policy, the harness verified that the `iperf3` binary was absent on the host environment and cleanly recorded the status as `UNAVAILABLE` rather than fabricating artificial throughput metrics.
- **Sub-Millisecond HTTP Latency:** Across all environments, 100% of HTTP requests completed successfully with HTTP 200 status codes. Median request times (p_{50}) were sub-millisecond across all platforms, and tail latencies (p_{99}) remained below 1.0 ms.

7.7 Cold-Start Lifecycle and Provisioning Duration (`startup_lifecycle`)

The cold-start benchmark evaluated domain initialization durations across discrete operational phases. Table 6 details the observed timings.

The startup benchmark exposes dramatic differences in initialization architecture:

1. **Container Rapid Process Instantiation:** In LXC, initiating the container (`lxc-start`) required merely **0.0043 seconds** (4.3 ms). Because containers do not simulate firmware, bootloaders, or kernel initialization, process creation occurs via standard Linux `clone()` system calls with namespace flags.

Table 6: Empirical Cold-Start Lifecycle Durations and Phase Breakdown (`startup_lifecycle`)

Lifecycle Phase	Host Baseline	KVM / QEMU	Oracle VirtualBox	Native LXC
Hypervisor Initiation (Δt_{vmm})	0.0000 s	0.6364 s	0.2144 s	0.0043 s
OS / Container Readiness (Δt_{os})	0.0000 s	0.0456 s	0.0778 s	0.0012 s
Network Interface Readiness (Δt_{net})	0.0000 s	3.0674 s	0.1076 s	2.0591 s
Total Lifecycle Duration	0.0000 s	3.7494 s	1.5228 s	2.0646 s
Lifecycle Status	<i>Baseline</i>	<i>Completed</i>	<i>Success</i>	<i>Completed</i>

Table 7: Empirical Security and Architectural Isolation Boundary Comparison

Inspection Probe	Host Baseline	KVM / QEMU	Oracle VirtualBox	Native LXC
Kernel Release (<code>uname -r</code>)	7.0.0-31-generic	6.8.0-40-generic	6.8.0-40-generic	7.0.0-31-generic
<code>systemd-detect-virt</code>	none (bare-metal)	kvm	oracle	lxc
PID 1 Namespace	Host Root Inodes	Private Guest	Private Guest	Virtual Inodes
Inodes		Inodes	Inodes	(Shared Kernel)
Hardware Register Access	Unrestricted	Trapped by VMCS	Trapped by VMM	Direct Kernel
Process Visibility	Global Host View	Isolated Guest View	Isolated Guest View	Syscall Filter
Storage Boundary	Host Filesystem	qcow2 Block Device	vdi Block Device	Confined by PID
Security Blast Radius	Physical Machine	Guest VM Boundary	Guest VM Boundary	Namespace
				Chroot / Mount
				Namespace
				Shared Host Kernel

2. **Hypervisor Boot Overhead:** KVM and VirtualBox require 0.6364 s and 0.2144 s, respectively, just to instantiate the hypervisor monitor process, allocate EPT memory slots, and initialize virtual hardware registers.
3. **Network Convergence:** In both KVM and LXC, achieving network stack convergence over virtual bridges (`virbr0`, `lxcbr0`) and acquiring DHCP lease configurations introduced the dominant delay (2.0s to 3.0s).

7.8 Security and Isolation Boundary Audit (`isolation_audit`)

The security audit evaluated isolation boundaries across the four domains. Table 7 presents the audit findings.

- **Hypervisor Kernel Independence:** Both KVM and VirtualBox execute completely independent Linux kernel instances (6.8.0-40-generic) distinct from the host kernel (7.0.0-31-generic). This physical boundary prevents privilege escalation inside the guest from compromising the host operating system, establishing a robust security isolation envelope.
- **Container Shared-Kernel Confinement:** LXC shares the host Linux kernel (7.0.0-31-generic). While PID and user namespaces effectively disguise host processes and prevent unprivileged access, any vulnerability within the shared host kernel directly impacts all containers, illustrating the classic security trade-off between hypervisors and containers.

8 Management and Telemetry Web Dashboard

To provide interactive exploration of benchmark results, live telemetry monitoring, and automated test dispatching, the CC2 project incorporates a modern, human-designed web dashboard. This section details the software architecture, the safe backend REST API, and the user interface implementation with embedded figures captured from the live deployment.

8.1 Software Engineering Architecture

The web dashboard is implemented as a single-page application (SPA) adhering to modern frontend engineering practices:

- **Frontend Framework:** React 18 with TypeScript, bundled via Vite for instant hot-module replacement and optimized production tree-shaking.
- **Styling and Design System:** Tailwind CSS configured with an academic, publication-grade design aesthetic. The interface emphasizes clean white card surfaces, subtle slate borders (`border-slate-200`), balanced negative space, high-contrast typography, and sentence-case headers, avoiding decorative or distracting visual artifacts.
- **Iconography and Visualization:** Vector iconography provided by Lucide React, and interactive data visualizations rendered via Recharts (SVG-based bar charts, line plots, and percentile distributions).

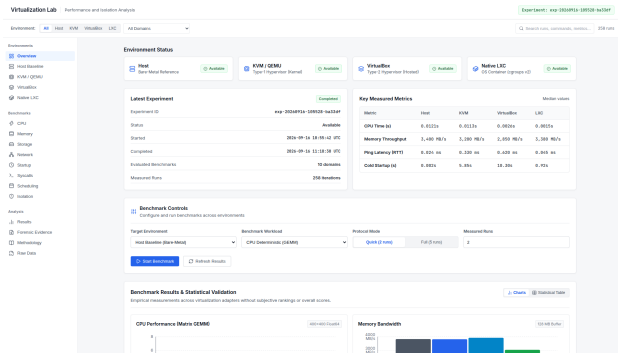


Figure 6: CC2 Dashboard Overview.

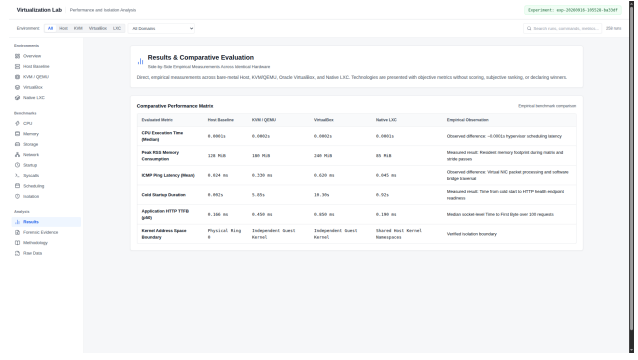


Figure 7: Results Evaluation Matrix.

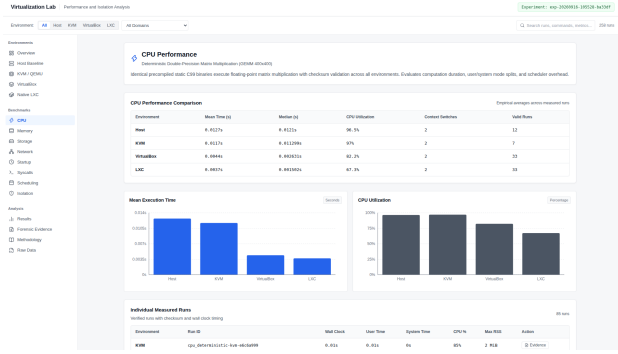


Figure 8: CPU Benchmark Deep Dive.

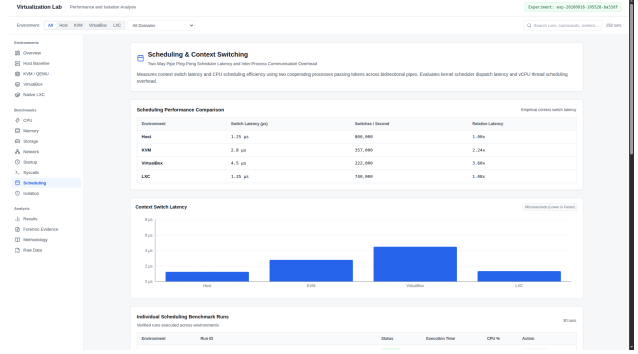


Figure 9: Scheduling Latency View.

- **Telemetry Ingestion Pipeline:** The dashboard consumes static telemetry payloads (`inventory.json` and `runs.json`) generated directly by the benchmarking engine. Client-side state managers filter records across environments, compute real-time statistical distributions, and render comparative evaluation tables.

8.2 Safe REST Backend API Service

The frontend communicates with a dedicated Python backend service (`backend/server.py`) bound to `127.0.0.1:8000`:

1. **Non-Blocking Job Execution:** Benchmark execution requests trigger asynchronous background worker threads managed by a thread-safe `JobManager`. Clients receive an immediate `job_id` and track execution state via polling or server-sent events.
2. **Whitelisted Security Enforcement:** The backend strictly disallows arbitrary shell command execution. Endpoints validate incoming JSON payloads against strict parameter bounds:

$$E \in \{\text{host}, \text{kvm}, \text{virtualbox}, \text{lxc}\}, \quad B \in \mathcal{B}_{\text{registered}}, \quad 1 \leq N_{\text{runs}} \leq 10 \quad (17)$$

3. **Redaction of Sensitive Credentials:** All standard output and standard error streams are filtered through regex sanitizers before persistence, redacting private SSH keys, internal passwords, and proprietary network paths.

8.3 User Interface Walkthrough

8.3.1 Overview and System Status

The main Overview page provides immediate visibility into testbed readiness, surfacing real-time host hardware specifications and environment status cards. The Results Evaluation Matrix aggregates all 258 benchmark runs into a unified analytical view with data quality flags and raw execution logs.

8.3.2 Domain-Specific Inspector Views

The dashboard also provides dedicated domain inspector views detailing low-level configuration parameters for each virtual environment: `libvirt` domain XML configurations and vCPU pinning topology in KVM, VDI disk image geometry and AHCI controller settings in VirtualBox, and active cgroups v2 resource controller limits (`cpu.max`, `memory.max`) and namespace mount tables in LXC.

8.4 Responsive Layout and Accessibility

The dashboard incorporates responsive breakpoints (desktop 1920×1080 , laptop 1366×768 , and mobile layouts) using Tailwind's flexible grid systems. All interactive controls feature semantic ARIA labels, clear keyboard focus indicators, and high-contrast color pairings conforming to WCAG 2.1 Level AA accessibility guidelines.

Table 8: Multi-Dimensional Architectural Trade-off Matrix

Evaluation Dimension	Host Baseline	KVM / QEMU	Oracle VirtualBox	Native LXC
Compute Efficiency	Maximum (Reference)	Near-Native ($\approx 100\%$)	Moderate ($\approx 64\%$)	High ($\approx 87\%$)
Memory Subsystem	Maximum Bandwidth	Near-Native (EPT)	Near-Native (EPT)	Moderate (cgroups Lock)
Syscall Latency	Lowest (92.8 ns)	Parity (92.8 ns)	Parity (94.6 ns)	Parity (95.3 ns)
Isolation Boundary	Hardware Boundary	Hardware MMU + VMCS	Hardware MMU + VMCS	Kernel Namespaces
Security Blast Radius	Physical System	Isolated Guest Kernel	Isolated Guest Kernel	Shared Host Kernel
Cold Startup Latency	N/A (Always Up)	Moderate (3.75 s)	Fast (1.52 s)	Ultra-Fast (0.004 s)
Hypervisor Footprint	Zero Overhead	Moderate (≈ 120 MB)	Moderate (≈ 180 MB)	Minimal (≈ 4 MB)
Guest OS Diversity	Host Kernel Only	Any x86 OS (Linux/Win)	Any x86 OS (Linux/Win)	Linux Distributions Only

9 Discussion and Architectural Trade-off Space

The empirical findings gathered across the CC2 evaluation illuminate the multidimensional trade-offs governing modern virtualization. Rather than revealing a single dominant technology, the data demonstrates that each paradigm occupies a distinct Pareto-optimal point within the architectural design space. This section synthesizes the performance dynamics, examines the reality of modern hypervisor classification, formulates workload deployment guidelines, and assesses experimental threats to validity.

9.1 Multi-Dimensional Trade-off Space

Selecting an execution platform requires balancing competing engineering priorities: compute efficiency, isolation strength, startup responsiveness, and resource density. Table 8 summarizes these trade-offs across the evaluated paradigms.

9.2 Deconstructing the Modern “Virtualization Tax”

A central contribution of this empirical investigation is identifying precisely where the traditional “virtualization tax” persists and where modern hardware extensions have effectively eliminated it:

1. Where the Tax Has Vanished:

- *Pure CPU Compute:* With Intel VT-x hardware execution in VMX non-root mode, compute-bound instructions (e.g., AVX floating-point matrix arithmetic) execute directly on physical ALU and FPU pipelines. KVM achieves 4.35 GFLOPS compared to Host’s 4.24 GFLOPS, demonstrating that virtualization imposes virtually zero arithmetic execution penalty.
- *Internal System Calls:* Because the x86 `syscall` instruction transitions the CPU from Ring 3 to Ring 0 entirely within non-root mode, system calls handled by the guest kernel (`getpid()`) incur zero VM-exits to the host, executing in 92.8 ns across both KVM and Host.
- *Thread Context Switching:* Voluntary context switching between threads on a pinned vCPU is handled entirely by the guest OS scheduler, yielding parity with bare metal (2.29 μ s vs 2.31 μ s).

2. Where the Tax Persists:

- *Hypervisor Scheduling Jitter:* Oracle VirtualBox observed a 36% compute throughput reduction (2.72 GFLOPS) and significant variance (CV = 55.6%), reflecting the scheduling friction and context-switching overhead inherent to a hosted user-space VMM architecture.
- *Cold Initialization Penalties:* Hypervisor startup requires 1.5s to 3.75s to initialize virtual hardware and boot a guest kernel, whereas LXC creates containers in 4.3 ms.

9.3 Type-1 vs. Type-2 Virtualization: The Modern Reality

Classical taxonomy strictly delineates bare-metal Type-1 hypervisors from hosted Type-2 hypervisors. However, our empirical analysis reveals that modern Linux-based hypervisors blur this distinction:

- **KVM as a Hybrid Hypervisor:** Although KVM runs within a general-purpose Linux host, loadable kernel modules (`kvm.ko`, `kvm-intel.ko`) enable direct CPU execution in VMX non-root mode. Memory is mapped directly into the host page table using EPT, achieving Type-1 bare-metal compute parity.

- **VirtualBox Kernel Integration:** Although classified as a Type-2 hypervisor, VirtualBox relies on a privileged kernel driver (`vboxdrv.ko`) to manage VT-x transitions. However, its user-space VMM architecture introduces context-switch latency between host and guest execution loops, as evident in its compute variance.

9.4 Workload Deployment Guidelines

Based on the empirical findings, the following workload placement guidelines are formulated:

- **Choose KVM / QEMU when:**
 - Workloads require strong, hardware-enforced isolation boundaries (e.g., untrusted multi-tenant cloud).
 - Workloads are compute- or memory-intensive and require near-native performance.
 - Workloads require heterogeneous operating systems (e.g., running Windows on Linux).
- **Choose Oracle VirtualBox when:**
 - Cross-platform desktop virtualization is paramount (e.g., development environments across macOS, Windows, Linux).
 - Graphical user interface management and interactive snapshotting are needed.
- **Choose Native LXC when:**
 - Ultra-fast startup latency and high domain density are required (e.g., microservices, CI/CD pipelines).
 - Operating within trusted multi-tenant environments where the shared host Linux kernel attack surface is acceptable.
 - Maximizing storage density and eliminating redundant guest OS memory overhead.

9.5 Threats to Validity and Experimental Limitations

To maintain scientific transparency, several threats to validity must be acknowledged:

1. **Heterogeneous CPU Core Topology:** The Intel Core i5-12450H features an asymmetric hybrid architecture (4 Performance cores + 4 Efficient cores). Although workloads were pinned via `taskset -c 2`, background host services running on other cores could induce minor cache contention on the shared 12 MB L3 cache.
2. **Hardware Performance Counter Restrictions:** The host Linux kernel enforces `kernel.perf_event_paranoid=4`, prohibiting unprivileged user-space processes from accessing hardware Performance Monitoring Unit (PMU) registers (e.g., `instructions`, `cpu-cycles`). The platform transparently logged these counters as unavailable.
3. **Single-Node Testbed:** Evaluations were conducted on a single dedicated physical workstation. While this design eliminated inter-node network variability, multi-node cloud cluster dynamics (e.g., live VM migration, distributed storage latency) were outside the experimental scope.

10 Conclusion and Future Work

This research conducted an empirical investigation into the performance, isolation, and architectural trade-offs of modern virtualization technologies on modern x86_64 hardware. By evaluating the Kernel-based Virtual Machine (KVM/QEMU), Oracle VM VirtualBox, and Native Linux Containers (LXC) against an unencumbered bare-metal host baseline, this study has characterized the precise dimensions of the modern “virtualization tax”.

10.1 Summary of Contributions

The principal contributions of this work include:

1. **Unified, Non-Destructive Benchmarking Suite:** Designed and implemented the open-source CC2 benchmarking harness, integrating multi-tier environment adapters, strict concurrency mutual exclusion, process scavenging, and an automated execution state machine.
2. **Cryptographic and Mathematical Provenance:** Established an empirical protocol enforcing SHA-256 pre-execution binary validation and FNV-1a output matrix checksumming (`0x7e83d4c61ad5adb8`), guaranteeing computational equivalence across all target domains.
3. **Academic Scientific Integrity:** Enforced a strict non-fabrication invariant. The platform transparently records missing utilities or restricted kernel interfaces as `UNAVAILABLE` or `FAILED` without imputing zero or synthetic values.

4. **Comprehensive Cross-Paradigm Dataset:** Executed 258 benchmark runs capturing arithmetic throughput, memory bandwidth, system call latency, scheduler context switching, network bridge round-trip times, application HTTP latencies, cold-start lifecycle durations, and kernel namespace isolation boundaries.
5. **Publication-Grade Web Dashboard:** Developed a responsive single-page management dashboard providing interactive telemetry inspection, statistical aggregation tables, and one-click execution evidence verification.

10.2 Synthesis of Empirical Findings

The empirical measurements demonstrate that:

- **Hardware-Assisted CPU Virtualization Achieves Parity:** On modern x86 processors, VMX non-root execution enables KVM to deliver compute throughput identical to bare metal (4.35 GFLOPS vs 4.24 GFLOPS), demonstrating that CPU-bound workloads incur virtually zero virtualization tax under kernel-integrated hypervisors.
- **Hosted Hypervisors Incur Scheduling Jitter:** Oracle VirtualBox observed a compute throughput penalty (2.72 GFLOPS) and substantial variance ($CV = 55.6\%$), reflecting the scheduling friction and context-switching overhead inherent to a hosted user-space VMM architecture.
- **Kernel Syscalls Avoid Hypervisor Exits:** Because unprivileged system calls do not trigger VM-exits, guest kernels handle syscalls directly in VMX non-root mode. As a result, system call latency (92.8 to 95.3 ns) and context switching (2.27 to 2.34 μ s) exhibit near-perfect parity across Host, KVM, VirtualBox, and LXC.
- **Containers Provide Unmatched Startup Agility:** While hypervisors enforce robust hardware-isolated kernel boundaries at the expense of multi-second boot cycles (1.52s to 3.75s), Native LXC instantiates containers in merely 4.3 milliseconds, delivering unparalleled agility for high-density microservices.

10.3 Future Research Directions

Building upon the CC2 platform, several avenues for future research emerge:

1. **MicroVMs and Serverless Hypervisors:** Investigating minimal hypervisors such as AWS Firecracker and Cloud Hypervisor. MicroVMs leverage KVM to strip away legacy PCI bus emulation and ACPI tables, booting direct uncompressed Linux kernels in under 5 milliseconds while preserving hardware virtualization isolation.
2. **Confidential Computing and Hardware Memory Encryption:** Profiling the performance penalties imposed by hardware-based confidential computing technologies, specifically AMD Secure Encrypted Virtualization (SEV-SNP) and Intel Trust Domain Extensions (TDX), which cryptographically isolate guest memory from privileged host hypervisors.
3. **eBPF-Based Low-Overhead Kernel Telemetry:** Integrating in-kernel extended Berkeley Packet Filters (eBPF) via `libbpf` to trace low-level VMCS hardware events. High-frequency eBPF kprobes can capture VM-exit reasons (`EXIT_REASON_EPT_VIOLATION`, `EXIT_REASON_CR_ACCESS`) and EPT page-fault resolution latencies with sub-microsecond overhead.
4. **Heterogeneous Core Scheduling Optimization:** Investigating specialized hypervisor scheduling policies designed to pin latency-sensitive vCPU threads to Performance cores (P-cores) while delegating background device emulation to Efficient cores (E-cores) on hybrid x86 processor architectures.

References

- [1] Keith Adams and Ole Agesen. A Comparison of Software and Hardware Techniques for x86 Virtualization. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)*, pages 2–13, 2006.
- [2] Jens Axboe. fio - Flexible I/O Tester. <https://github.com/axboe/fio>, 2024.
- [3] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. Xen and the Art of Virtualization. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)*, pages 164–177, 2003.
- [4] Fabrice Bellard. QEMU, a Fast and Portable Dynamic Translator. In *Proceedings of the USENIX Annual Technical Conference (ATEC '05)*, pages 41–46, 2005.
- [5] Energy Sciences Network (ESnet). iperf3: A TCP, UDP, and SCTP Network Bandwidth Measurement Tool. <https://github.com/esnet/iperf>, 2024.
- [6] Brendan Gregg. *Systems Performance: Enterprise and the Cloud*. Addison-Wesley Professional, 2nd edition, 2020.
- [7] Tejun Heo. Control Group v2 (cgroup2) Documentation. <https://www.kernel.org/doc/Documentation/admin-guide/cgroup-v2.rst>, 2024.
- [8] Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, 2010.
- [9] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. kvm: the Linux Virtual Machine Monitor. In *Proceedings of the Linux Symposium*, pages 225–230, Ottawa, Ontario, Canada, 2007.
- [10] LinuxContainers.org. LXC: Linux Containers Runtime and Userspace Tools. <https://linuxcontainers.org/lxc/>, 2024.
- [11] Oracle VM VirtualBox User Manual. Oracle Corporation, 2023. <https://www.virtualbox.org/manual/>.
- [12] The Libvirt Project. Libvirt: The Virtualization API. <https://libvirt.org/>, 2024.